Підготовка до заняття 6

Data Ingestion & Preprocessing для AI

Як читати цей матеріал

Це навчальний текст, а не конспект: він пояснює тему з нуля і спирається лише на те, що було на заняттях 1–5 (токени, контекстне вікно, виклик LLM через API, structured output, промпти). Читайте розділи по порядку: кожен наступний використовує попередній. Усі приклади взято з корпусу, з яким працюємо на практиці, тож на занятті ви впізнаєте ці документи.

Наприкінці кожного розділу є блок **«Головне»** з тим, що варто запам'ятати, і посилання **«Глибше»** для тих, хто хоче розібратися детальніше. Посилання необов'язкові. Розділ 9 — питання для самоперевірки: якщо відповідаєте на них без підглядання, ви готові до заняття. Орієнтовний час читання — 60–75 хвилин.

План

1. Навіщо AI-інженеру ingestion
проблема, визначення, місце в курсі

2. Як влаштований пайплайн
етапи, batch vs streaming, інкрементальність

3. Витягування тексту
PDF, скани й OCR, HTML, DOCX, кодування

4. Очищення і нормалізація
Unicode, колонтитули, переноси, PII

5. Дедуплікація
хеші, точні й майже однакові копії

6. Chunking
що таке чанк, розмір, стратегії

7. Метадані, доступ і аудит
що зберігати поряд з текстом

8. Версіонування і продакшн-патерни
manifest, шари даних, терміни

9. Перевірте себе
питання для самоперевірки

10. Перед заняттям
репозиторій і домашня частина практики

Що ви вмітимете після заняття

- обирати патерн ingestion (batch чи streaming) під джерело даних;
- витягувати текст із PDF, DOCX, HTML і текстових файлів і бачити, де якість ламається;
- будувати очищення й дедуплікацію, які не псують зміст;
- проєктувати chunking і метадані під майбутній пошук (заняття 7–8).

РОЗДІЛ 1

1 Навіщо AI-інженеру ingestion

> Після розділу: ви розумієте, яку проблему вирішує ingestion, і можете пояснити, чому це окрема інженерна задача, а не «просто прочитати файли».

1.1 Проблема: модель не знає ваших документів

На занятті 2 ми розбирали, як навчається модель: вона бачила величезний корпус публічних текстів до певної дати (knowledge cutoff). Внутрішніх документів вашої компанії — політики відпусток, інструкції з VPN, звіту за квартал — у тих даних не було. Тому на питання «скільки днів на тиждень я можу працювати з дому?» модель або чесно скаже, що не знає, або, що гірше, вигадає правдоподібну відповідь (галюцинація з заняття 4).

Рішення — дати моделі потрібний текст у контексті запиту: «ось уривок з політики, відповідай за ним». Але контекстне вікно обмежене, а документів у компанії тисячі. Тому потрібна система, яка спершу знайде кілька релевантних шматків тексту, а потім передасть їх моделі. Це RAG (retrieval-augmented generation), ми будуватимемо його на занятті 8. А сьогоднішня тема — те, з чого RAG починається: як перетворити купу різнорідних файлів на чистий, структурований текст, у якому можна шукати.

1.2 Що таке data ingestion

Data ingestion (дослівно «поглинання даних») — процес, який забирає документи з джерел (папки, Google Drive, Confluence, база даних, сайт), витягує з них текст, приводить його до порядку і записує у вигляді, придатному для пошуку. **Preprocessing** — та частина цього процесу, що відповідає за очищення і підготовку тексту.

Аналогія: бібліотекар

Бібліотека отримує книжки з різних джерел: нові, старі, ксерокопії, дублікати. Бібліотекар не ставить їх на полиці як є. Він прибирає зайві копії, відкладає пошкоджені на реставрацію, розставляє решту за розділами і заводить на кожну картку: назва, розділ, рік, хто може брати. Без цієї роботи читач не знайде потрібне, навіть якщо книжка є. Ingestion-пайплайн робить те саме для документів, а «картка» — це метадані, про які йтиметься в розділі 7.

1.3 Чому це інженерна задача

Здавалося б, прочитати файл просто. Але на реальних даних ламається майже все. Ось що повертає найпростіший код на документах нашої практики:

- з HTML-сторінки інтранету разом зі статтею приходять меню, текст cookie-банера «We use cookies… Accept all cookies» і JavaScript;
- з PDF на кожній сторінці повторюються шапка «Northwind Logistics — Internal» і рядок «Page 2 of 3», а слова розірвані переносом: `secur-` / `ity`;
- скан підписаного договору — це PDF без жодного символу тексту, лише картинка;
- старий файл у кодуванні Windows-1251 замість «Інструкція з безпеки» читається як `������������ ������`;

- та сама політика лежить у двох папках, а політика віддаленої роботи — у двох версіях з різними правилами.

Кожна така помилка не зникає далі, а множиться. Якщо cookie-банер потрапив у пошуковий індекс, то на питання «як прийняти умови?» пошук знайде саме його. Якщо в індексі лишилась стара версія політики, модель отримає два суперечливі правила і може процитувати застаріле. Жоден промпт цього не виправить: модель відповідає за тим текстом, який їй дали. Принцип відомий як **garbage in, garbage out**: якість відповіді обмежена якістю даних на вході.

1.4 Місце в курсі

Блок 2 курсу будує одну систему за чотири кроки. Сьогоднішній результат — чисті чанки з метаданими — стане входом для всіх наступних занять:



ГОЛОВНЕ

Модель не знає ваших документів, тому текст треба знайти і передати їй у контексті. Ingestion готує цей текст: забирає файли, витягує, чистить, прибирає дублі, ріже на шматки й підписує. **Помилки на вході не виправляються промптом**, вони лише множаться далі.

РОЗДІЛ 2

2 Як влаштований пайплайн

> Після розділу: ви знаєте етапи ingestion, що на вході й виході кожного, і чим batch відрізняється від streaming.

2.1 Етапи

Пайплайн — послідовність кроків, де вихід одного кроку є входом наступного. Типовий ingestion складається з семи етапів:



Подивімося, як проходить цей шлях один документ — сторінка інтранету `handbook/onboarding.html`:

Етап	Що з документом
Забрати	1 836 байтів HTML: стаття, меню, cookie-банер, скрипти, футер
Витягти	лише вміст `<main>`; заголовки `<h2>` стають рядками `## Day 1`, пункти списку — `- …`
Очистити	прибрано нерозривний пробіл (` `) і невидимий символ zero-width space
Дедуплікація	копій немає — документ іде далі
Нарізати	5 чанків: по одному на розділ «Before your first day», «Day 1», «Week 1»…
Метадані	до кожного чанка: джерело, назва, розділ, хто може читати, хеш, дата обробки
Записати	рядки в `chunks.jsonl`, запис у manifest і журнал подій

2.2 Batch vs streaming

Перше архітектурне рішення — **коли** запускати пайплайн. Є два основні патерни.

Batch (пакетна обробка): пайплайн запускається за розкладом і обробляє все джерело за раз. Наприклад, щоночі о 02:00 забирає всі документи з Confluence. Це просто: один скрипт, легко перезапустити, легко зрозуміти, що сталось. Ціна — затримка: документ, змінений о 10:00, з'явиться в пошуку лише наступного ранку.

Streaming (потокова, або event-driven обробка): пайплайн реагує на кожну подію окремо. Джерело повідомляє «документ змінено» через **webhook** (HTTP-запит від джерела до вашого сервісу) або через чергу повідомлень. Для баз даних використовують **CDC** (change data capture) — механізм, що читає журнал змін бази і передає кожну вставку, зміну чи видалення. Затримка — секунди чи хвилини. Ціна — складність: події можуть прийти двічі або не в тому порядку, видалення треба обробляти окремо, а сервіс має працювати постійно.

	Batch	Streaming
Як запускається	за розкладом, усе джерело	на кожну зміну
Затримка	години	секунди–хвилини
Складність	низька	висока: дублі подій, порядок, видалення
Приклади	політики, довідники, архів	тікети підтримки, ціни, статуси замовлень

Як обрати: спитайте власника даних, **за скільки часу зміна має з'явитися в пошуку**. Якщо «до ранку достатньо» — batch. Якщо «оператор має бачити новий тікет за хвилину» — streaming. Більшість внутрішніх баз знань починають з batch.

2.3 Інкрементальність та ідемпотентність

Batch-пайплайн, який щоночі переобробляє всі 50 000 документів, марнує час і гроші, бо змінилось, може, 30. **Інкрементальний** запуск обробляє лише нове і змінене. Для цього пайплайн зберігає **manifest** — таблицю «файл → відбиток його вмісту» з минулого запуску. Відбиток — це хеш (детально в розділі 5): якщо хеш файлу не змінився, файл пропускаємо.

```
# output/manifest.json (фрагмент з практики)
"it/vpn_setup.docx": {
  "raw_hash": "c8e2cf79…",   # хеш байтів файлу: чи змінився файл?
  "status": "ok",
  "chunks": 4
}
```

Ідемпотентність — властивість операції давати той самий результат, скільки б разів її не повторили. Для ingestion це означає: якщо пайплайн упав посередині й ви запустили його знову, в індексі не з'являться дублікати чанків. Цього досягають стабільними ідентифікаторами: чанк `it/vpn_setup.docx#2` при повторному запуску **замінює** сам себе, а не додається поруч.

ГОЛОВНЕ

Пайплайн — це сім етапів: забрати → витягти → очистити → дедуплікація → нарізати → метадані → записати. **Batch** простіший, **streaming** свіжіший; вибір залежить від того, як швидко зміна має потрапити в пошук. Інкрементальний запуск обробляє лише змінене, ідемпотентний — не дублює результат при повторі.

Глибше: LlamaIndex: Ingestion Pipeline — як ці етапи виглядають у популярному фреймворку.

РОЗДІЛ 3

3 Витягування тексту

> Після розділу: ви розумієте, чому кожен формат ламається по-своєму, вмієте відрізнити PDF з текстом від скану і знаєте, що таке кодування.

3.1 Файл — це ще не текст

Для людини PDF, DOCX і вебсторінка з однаковим змістом виглядають однаково. Для програми це три зовсім різні структури даних. Модель же приймає лише текст (і токени з нього, як ми бачили на занятті 2). Тому перший етап — дістати з кожного формату текст і при цьому **не втратити структуру**: де заголовок, де список, де таблиця. Структура знадобиться в розділі 6, щоб різати текст за змістом.

3.2 PDF з текстовим шаром

PDF створювався для друку, а не для читання програмою. Усередині він зберігає не абзаци, а інструкції на кшталт «намалюй символи “Remote days” у точці (50, 712) шрифтом Helvetica 10». Бібліотека, що витягує текст, збирає ці фрагменти назад у рядки, вгадуючи порядок за координатами. Звідси типові проблеми:

- **колонтитули** — шапка й номер сторінки є на кожній сторінці і потрапляють у текст;
- **переноси** — рядок закінчується там, де закінчилась ширина сторінки, а не речення; довгі слова розірвані дефісом;
- **колонки й таблиці** — дві колонки можуть злитися в один рядок, клітинки таблиці — розсипатися.

Ось реальний вивід бібліотеки `pypdf` для другої сторінки політики віддаленої роботи з практики:

```
Northwind Logistics — Internal       ← шапка, повторюється на кожній сторінці
Effective 2026-01-15
3. Approval
Remote days are requested in the HR portal at least one working day in advance. The team
lead approves or declines the request within one working day. ← рядок розірвано посеред
речення
…
Page 2 of 3                          ← номер сторінки
```

```
# як отримати текст PDF посторінково
from pypdf import PdfReader
reader = PdfReader("data/raw/policies/remote_work_policy.pdf")
pages = [page.extract_text() or "" for page in reader.pages]
text = "\f".join(pages)    # \f (form feed) — роздільник сторінок, знадобиться для пошуку
колонтитулів
```

3.3 Скани і OCR

Скан — це фотографія сторінки, вкладена в PDF. Текстового шару в ньому немає, тож `extract_text()` поверне порожній рядок. Головна небезпека в тому, що **помилки не буде**: пайплайн «успішно» обробить документ і проіндексує нуль символів. Договір зникне з пошуку, і ніхто цього не помітить.

Тому скан треба **виявити**. Найпростіше правило: порахувати символи. Сторінка звичайного документа містить сотні символів; якщо в середньому на сторінку менше ~50, текстового шару фактично немає:

```
chars = sum(len(p.strip()) for p in pages)
needs_ocr = chars < 50 * len(pages)    # скан: позначити, а не індексувати порожнечу
```

OCR (optical character recognition, оптичне розпізнавання символів) — технологія, що «читає» текст із зображення. Безкоштовний варіант — Tesseract, хмарні — Google Document AI, AWS Textract, Azure Document Intelligence; сучасні vision-моделі (gpt-4o, Claude) теж уміють читати сторінки. OCR повільний, коштує грошей і помиляється: «0» замість «O», «rn» замість «m», особливо на кривих сканах. Тому його запускають **лише там, де текстового шару немає**, а не на всіх PDF підряд. На практиці ми не робимо OCR, а позначаємо скан статусом `ocr_required` — у реальній системі він пішов би в окрему чергу на розпізнавання.

3.4 HTML: стаття серед шуму

Вебсторінка — це стаття, загорнута в обгортку сайту: меню, cookie-банер, футер, скрипти аналітики. Цю обгортку називають **boilerplate** (шаблонний текст, однаковий на всіх сторінках). Якщо взяти весь текст сторінки, boilerplate потрапить у кожен документ сайту. Рішення — розібрати HTML як дерево тегів і викинути зайві гілки:

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html, "html.parser")
for tag in soup(["script", "style", "nav", "footer"]):    # меню, скрипти, футер
    tag.decompose()
for tag in soup.select("[id*=cookie], [class*=cookie]"):  # cookie-банер
    tag.decompose()
article = soup.find("main") or soup.body                  # сама стаття
```

Заголовки `<h1>` – `<h3>` варто перетворити на рядки `#`, `##`, `###` (як у Markdown). Так текст зберігає структуру, і далі ми зможемо різати його за розділами. Для довільних сайтів, де немає `<main>`, існують бібліотеки, що вгадують основний вміст автоматично, наприклад trafilatura.

3.5 DOCX: структура в стилях

Word-документ усередині — це архів з XML-файлами. Важливо, що структуру він зберігає через **стилі**: заголовок — це абзац зі стилем «Heading 1», пункт нумерованого списку — абзац зі стилем «List Number». Таблиці лежать окремо від абзаців. Якщо взяти лише текст абзаців, заголовки стануть звичайними рядками, нумерація зникне, а таблиці пропадуть повністю. Тому витягуємо з урахуванням стилів і перетворюємо таблицю на рядки Markdown:

НАІВНО: лише абзаци

```
VPN setup
All remote connections…
Install the client
Open the Software Center…
Servers
Choose the server closest…
(таблиці немає зовсім)
```

ЗІ СТРУКТУРОЮ

```
# VPN setup
All remote connections…
## Install the client
1. Open the Software Center…
## Servers
| Server | Location | Use for |
|---|---|---|
| vpn-kyiv… | Kyiv | Default… |
```

3.6 Текстові файли і кодування

Комп'ютер зберігає текст як байти — числа від 0 до 255. **Кодування** — це таблиця, яка каже, яким символам відповідають байти. Сучасний стандарт — **UTF-8**: він уміщує всі мови світу. Але старі файли, створені в Windows українською чи російською, часто збережені в **Windows-1251 (cp1251)** — кодуванні, де той самий байт означає іншу літеру. Якщо прочитати cp1251-файл як UTF-8, вийде або помилка, або символи-заглушки; якщо як Latin-1 — «кракозябри» (по-англійськи **mojibake**):

```
правильно (cp1251):  Інструкція з безпеки на складі
прочитано як UTF-8:  ########## # ####### ## #####
прочитано як Latin-1: ²íñòðóêö³ÿ ç áåçïåêè íà ñêëàä³
```

Кодування у файлі ніде не записане, його доводиться вгадувати за статистикою байтів. Цим займається бібліотека `charset-normalizer`. Після визначення текст зберігаємо вже в UTF-8, щоб далі по пайплайну кодування було одне.

```
from charset_normalizer import from_bytes
best = from_bytes(path.read_bytes()).best()   # вгадує кодування
text = str(best)                               # тепер це звичайний рядок Python
```

Ще одна дрібниця — **BOM** (byte order mark), невидимий символ `U+FEFF`, який деякі редактори Windows ставлять на початок файлу. Він не видно на екрані, але змінює хеш і може потрапити в текст. Його прибираємо.

3.7 Інструменти для продакшну

На практиці ми пишемо витягування самі, щоб побачити, що відбувається всередині. У продакшні для складних документів (наукові PDF з колонками, звіти з таблицями, презентації) беруть готові

інструменти: **unstructured** і **Docling** розпізнають макет сторінки, **Apache Tika** читає сотні форматів, **LlamaParse** та інші сервіси використовують vision-моделі для складних PDF.

Формат	Що ламається	Чим витягувати
PDF з текстом	колонтитули, переноси, колонки, таблиці	pypdf, pdfplumber
Скан, фото	тексту немає взагалі	OCR: Tesseract, хмарний OCR, vision-моделі
HTML	меню, банери, футер, скрипти	BeautifulSoup, trafilatura
DOCX	втрачаються заголовки, списки, таблиці	python-docx
TXT, CSV	кодування, BOM	charset-normalizer

ГОЛОВНЕ

Кожен формат ламається по-своєму, тому витягування пишуть окремо для кожного. **Зберігайте структуру** (заголовки, списки, таблиці): без неї не вийде добре нарізати текст. Скан без текстового шару треба **виявити й позначити**, а не проіндексувати порожнечу. Кодування треба визначити, а не вважати, що все в UTF-8.

Глибше: pypdf · Tesseract OCR · unstructured · Docling · Joel Spolsky: про Unicode і кодування

РОЗДІЛ 4

4 Очищення і нормалізація

> Після розділу: ви знаєте, навіщо приводити текст до однієї форми, вмієте прибрати колонтитули й переноси і розумієте, що чистити не можна.

4.1 Навіщо нормалізувати

Нормалізація — приведення тексту до однієї канонічної (стандартної) форми. Мета проста: **однаковий зміст має виглядати однаково**. Від цього залежать три наступні етапи. Дедуплікація порівнює тексти: якщо копія відрізняється лише кінцями рядків, без нормалізації ми її не впізнаємо. Пошук порівнює слова: «e-mail» з нерозривним дефісом і «e-mail» зі звичайним — для програми різні рядки. Chunking рахує токени: сміття займає місце в чанку.

4.2 Unicode і NFKC

Unicode — стандарт, що дає кожному символу світу номер. Підступ у тому, що **один і той самий видимий символ можна записати різними номерами**:

Що бачимо	Як записано	Звідки береться
пробіл	`U+00A0` — нерозривний пробіл (у HTML ` `)	вебсторінки, Word
fi	`U+FB01` — лігатура «fi» одним символом	PDF з типографським шрифтом
(нічого)	`U+200B` — zero-width space, невидимий	копіювання з вебу, месенджери
１ ２ ３	повноширинні цифри	азійські розкладки, PDF

Стандарт Unicode описує способи нормалізації. Форма **NFKC** замінює «сумісні» символи звичайними: нерозривний пробіл стає пробілом, лігатура — двома літерами f і і. Невидимі символи NFKC не

прибирає, їх видаляємо окремо:

```
import re, unicodedata
text = unicodedata.normalize("NFKC", text)
text = text.replace("\r\n", "\n")                    # кінці рядків Windows → Unix
text = re.sub(r"[]", "", text)   # невидимі символи і BOM
```

4.3 Колонтитули і номери сторінок

Як автоматично відрізнити шапку від змісту? Шапка **повторюється на більшості сторінок**, а звичайний текст — ні. Тому рахуємо, на скількох сторінках зустрічається кожен рядок, і прибираємо ті, що є більш ніж на половині сторінок. Номери сторінок відрізняються («Page 1 of 3», «Page 2 of 3»), тож для них потрібен регулярний вираз: `^(page\s+)?\d+\s*(of|/)\s*\d+$`.

Підступ: на односторінковому документі «повторів» немає, і шапка лишиться. Тому правило про повтори працює лише для документів з кількох сторінок. Це нормально: жодне правило очищення не ідеальне, важливо знати його межі.

4.4 Переноси слів і розірвані рядки

Слово, розірване дефісом на межі рядка, склеюємо: дефіс, за яким іде перенос рядка і літера, видаляємо разом з переносом.

```
text = re.sub(r"(\w)-\n(\w)", r"\1\2", text)    # "secur-\nity" → "security"
```

Обережно: так само склеїться й справжнє складне слово, яке випадково опинилось на межі рядка (`e-\nmail` → `email`). Для пошуку це прийнятна втрата, для юридичного тексту — може бути ні. Окремо склеюємо рядки, які PDF розірвав посеред речення: якщо рядок довгий і не закінчується крапкою, а наступний не є заголовком чи пунктом списку, — це продовження того самого абзацу.

4.5 Що не можна «чистити»

Агресивне очищення шкодить так само, як відсутність очищення. Не видаляйте і не змінюйте:

- **числа, суми, дати, коди помилок** — саме про них найчастіше питають («який ліміт на готель?»);
- **таблиці** — рядок `| Hotel, per night | 120 EUR | Team lead |` має лишитися цілим рядком;
- **назви й ідентифікатори** — `vpn-kyiv.northwind.example` не можна «виправляти» орфографічно;
- **регістр і пунктуацію** в тексті, який піде в модель, — вони несуть зміст. Знижувати регістр можна лише для порівняння (хеш, дедуплікація), а не в самому тексті.

Практичне правило: **кожне правило очищення — окремий тест на реальному документі**. У практиці це `check.py`: він перевіряє і що сміття зникло, і що потрібний зміст лишився.

4.6 Персональні дані і секрети

Документи компанії містять персональні дані (PII — personally identifiable information: імена, телефони, паспортні дані) і секрети (паролі, API-ключі в інструкціях). Усе, що потрапило в індекс, може потрапити у відповідь моделі будь-кому, хто поставить вдале питання. Тому PII маскують або видаляють **на етапі ingestion, до індексу**: `+380 67 123 45 67` → `[PHONE]`. Покладатися на

інструкцію в промпті «не розкривай персональні дані» не можна: з заняття 4 ми знаємо, що промпт — не захист.

ГОЛОВНЕ

Нормалізація робить однаковий зміст однаковим: NFKC, один формат кінців рядків, без невидимих символів. Колонтитули знаходимо як рядки, що повторюються на сторінках. Переноси склеюємо. **Числа, таблиці й назви не чіпаємо**, а кожне правило перевіряємо тестом. PII прибираємо до індексу, а не в промпті.

Глибше: Python: unicodedata.normalize · Unicode: форми нормалізації (TR15) · Microsoft Presidio — пошук і маскування PII

РОЗДІЛ 5

5 Дедуплікація

> Після розділу: ви розумієте, що таке хеш, чим хеш файлу відрізняється від хешу змісту, і як знайти не лише точні копії, а й майже однакові версії.

5.1 Навіщо прибирати дублі

У будь-якій компанії документи копіюють: у спільну папку, в архів, «на всяк випадок». Дублі в індексі шкодять двома способами. По-перше, пошук повертає обмежену кількість результатів (top-k, наприклад 5 найкращих чанків). Якщо три з них — копії одного абзацу, корисних результатів лишається два. По-друге, **різні версії** одного документа дають моделі суперечливий контекст: у v1 політики «два дні з дому», у v2 — «три». Модель може процитувати будь-яку.

5.2 Що таке хеш

Хеш-функція перетворює дані будь-якого розміру на короткий рядок фіксованої довжини — «відбиток». Головні властивості: однакові дані завжди дають однаковий хеш, а зміна навіть одного символу дає зовсім інший хеш. Найпоширеніша — **SHA-256**: її відбиток — 64 шістнадцяткові символи. Порівнювати хеші набагато швидше, ніж порівнювати документи цілком.

```
import hashlib
hashlib.sha256("Remote Work Policy".encode()).hexdigest()   # '4b995597…' (64 символи)
hashlib.sha256("Remote Work policy".encode()).hexdigest()   # '3e233cbb…' — одна літера,
зовсім інший хеш
```

5.3 Хеш файлу vs хеш змісту

У практиці є файл `handbook/expense_policy.md` і його копія на спільному диску. Копію відкрили в іншому редакторі на Windows: з'явились BOM, кінці рядків CRLF, пробіли в кінці рядків, і два абзаци переформатовано в один рядок. Для людини зміст ідентичний. Але дивіться, що кажуть різні хеші:

Що хешуємо	Оригінал	Копія	Однакові?
байти файлу	`1ee5920d…`	`22a190bc…`	ні
текст після нормалізації	`f08f6d5b…`	`baa95e0e…`	ні — різне перенесення рядків
канонічна форма	`6eeb3732…`	`6eeb3732…`	**так**

Канонічна форма — текст, з якого прибрано все, що не впливає на зміст: усі пробіли й переноси згорнуто в один пробіл, регістр знижено. Хеш канонічної форми називають **content hash** (хеш змісту).

```
def content_hash(text):
    canonical = " ".join(text.split()).casefold()   # згорнути пробіли, знизити регістр
    return hashlib.sha256(canonical.encode()).hexdigest()
```

Зверніть увагу: це **два різні хеші з різними задачами**. Хеш байтів файлу відповідає на питання «чи змінився файл з минулого запуску?» — для інкрементальності з розділу 2. Хеш змісту відповідає на питання «чи це той самий документ?» — для дедуплікації.

5.4 Майже однакові документи

Хеш ловить лише точні копії: зміна одного слова — і хеш інший. А нам треба знайти й ситуацію, коли v1 і v2 політики відрізняються одним правилом. Для цього порівнюють не весь текст, а його шматочки.

Шинґли (shingles, «черепиця») — усі послідовності з k слів підряд. Наприклад, для k = 3 речення «may work remotely up to two days» дає шинґли: «may work remotely», «work remotely up», «remotely up to», «up to two», «to two days». Схожість двох документів міряють **коефіцієнтом Жаккара** (Jaccard):

```
Jaccard(A, B) = |спільні шинґли| / |усі різні шинґли обох документів|
```

0 — нічого спільного, 1 — ідентичні

Розберімо на мініприкладі. Речення v1: «may work remotely up to two days», v2: «may work remotely up to three days». Шинґли з 3 слів: у v1 — 5, у v2 — 5. Спільні: «may work remotely», «work remotely up», «remotely up to» — 3. Усього різних: 5 + 5 − 3 = 7. Jaccard = 3/7 ≈ 0.43. Одне змінене слово «зіпсувало» три шинґли, бо кожне слово входить у кілька шинґлів. На цілому документі одна зміна псує лише кілька шинґлів із сотень, тож схожість лишається високою: для двох версій політики в практиці Jaccard = **0.90**. Поріг «майже дубль» зазвичай 0.8–0.9.

Порівнювати кожну пару документів можна, коли їх сотні. Для мільйонів (наприклад, при підготовці даних для навчання моделей) використовують **MinHash + LSH** — алгоритми, що швидко знаходять схожі пари без перебору всіх. Embeddings із заняття 7 теж дозволяють шукати дублі за змістом, навіть перефразовані.

5.5 Яку версію лишити

Знайти майже-дубль — половина справи; треба вирішити, що з ним робити. Для точних копій неважливо, яку лишити. Для версій — зазвичай **новішу**: у практиці беремо дату «Effective …» з тексту, і v2 від 2026-09-01 витісняє v1 від 2026-01-15. Іноді правило інше: лишити документ з «офіційного»

джерела, а не зі спільної папки. Головне — щоб правило було явним і записаним у журнал (розділ 7), а не «яка першою трапилась».

ГОЛОВНЕ

Хеш змісту (від канонічної форми) знаходить точні копії, навіть із різними кінцями рядків і пробілами. **Шинґли + Jaccard** знаходять майже однакові версії. Хеш файлу і хеш змісту — різні інструменти: перший для інкрементальності, другий для дедуплікації. Для версій потрібне явне правило, яку лишити.

Глибше: datasketch: MinHash LSH · Jaccard index · MinHash

РОЗДІЛ 6

6 Chunking

> Після розділу: ви знаєте, що таке чанк, чому документи ріжуть на шматки, від чого залежить розмір і чим відрізняються основні стратегії.

6.1 Навіщо різати документи

Чанк (chunk, «шматок») — фрагмент документа, який зберігається, шукається і передається моделі як окрема одиниця. Чому не передавати документи цілком? Три причини:

1. **Контекстне вікно і вартість.** З заняття 3 пам'ятаємо: кожен токен у запиті оплачується. Щоб відповісти на питання про ліміт на готель, не потрібна вся політика витрат — достатньо таблиці лімітів. Десять сторінок замість одного абзацу — це в десятки разів дорожче і повільніше.
2. **Точність пошуку.** На занятті 7 кожен чанк перетвориться на embedding — вектор, що описує його зміст. Вектор цілого документа «розмазаний» між усіма його темами й погано відповідає конкретному питанню. Вектор короткого розділу про ліміти — точно про ліміти.
3. **Якість відповіді.** Модель краще працює з коротким релевантним контекстом, ніж з довгим, де потрібне речення загубилось серед іншого.

6.2 Розмір чанка

Розмір міряють у **токенах** (заняття 2), бо саме токени обмежують контекст і визначають вартість. На практиці рахуємо їх бібліотекою `tiktoken`. Розмір — завжди компроміс:

- **Замалий чанк** втрачає контекст. Чанк «Ліміт — 120 EUR, погоджує тімлід» без заголовка не каже, ліміт чого. Пошук не знайде його за питанням «скільки можна витратити на готель?».
- **Завеликий чанк** несе шум: разом з потрібним реченням у контекст іде сторінка нерелевантного тексту, а embedding стає «розмазаним».

Типові значення — 200–800 токенів. Правильний розмір для вашої задачі показує не інтуїція, а вимірювання якості пошуку (заняття 8–9). Наші демодокументи короткі, тому на практиці беремо 120 токенів, щоб різниця між стратегіями була помітна.

6.3 Фіксований розмір з перекриттям

Найпростіша стратегія: брати вікно з N токенів, потім наступне вікно і так далі. Щоб речення на межі не втратилось повністю, вікна **перекриваються** (overlap): кінець одного чанка повторюється на

початку наступного.

```
Upload the receipt to the expense tool within 30 days. Claims without a receipt are rejected.
Reimbursement is paid with the next salary. Alcohol, fines, personal …
```

Синій — чанк 1, помаранчевий — чанк 2, червоне — перекриття, яке є в обох.

Перевага — простота і передбачуваний розмір. Проблема — вікно не знає, що ріже. Ось реальний результат на політиці витрат (120 токенів, перекриття 20):

КІНЕЦЬ ЧАНКА 1

```
| Meals, per day | 45 EUR | Not needed |
| Training | 600 EUR per year | Team lead
and HR |
| Taxi | 30 EUR per trip |
```

ПОЧАТОК ЧАНКА 2

```
 600 EUR per year | Team lead and HR |
| Taxi | 30 EUR per trip | Not needed |

## How to submit
…
```

Рядок таблиці розрізано навпіл, а чанк 2 починається з «600 EUR per year» — без назви стовпців і без розділу. На питання «чи треба погоджувати таксі?» чанк 1 відповідає неповно, а чанк 2 не каже, що це взагалі таблиця лімітів витрат.

6.4 Chunking за структурою

Краща стратегія використовує структуру, яку ми зберегли під час витягування (розділ 3): спершу ділимо текст за заголовками на розділи, а потім усередині розділу набираємо цілі абзаци, поки вони вміщуються в ліміт токенів. Абзац і таблиця ніколи не ріжуться посередині. Таку стратегію називають **recursive** або **structure-aware**: спершу пробуємо різати по найбільших одиницях (розділ), і лише якщо не вміщується — по менших (абзац, речення).

```
# той самий документ, chunking за заголовками: таблиця ціла, чанк знає свій розділ
section: "Expense Policy > Limits"
| Category | Limit | Approval |
|---|---|---|
| Hotel, per night | 120 EUR | Team lead |
| Meals, per day | 45 EUR | Not needed |
| Training | 600 EUR per year | Team lead and HR |
| Taxi | 30 EUR per trip | Not needed |
```

На всьому корпусі практики різниця така: фіксоване вікно дало 15 чанків, з них 2 починаються посеред речення і 3 ріжуть рядки таблиць; chunking за заголовками дав 26 чанків без жодного такого розриву. Ціна — нерівні розміри: розділ з одного речення дає маленький чанк, а довгий абзац без заголовків може перевищити ліміт (ми свідомо не ріжемо абзац, бо зміст важливіший за рівний розмір).

6.5 Інші стратегії

Стратегія	Ідея	Коли
Semantic chunking	Межа чанка там, де змінюється тема: сусідні речення порівнюють через embeddings	довгі тексти без заголовків (стенограми, листи)
Parent-child	Шукають по малих чанках (точно), а моделі передають батьківський розділ (повно)	коли потрібні і точний пошук, і повний контекст
Contextual header	До тексту чанка дописують назву документа й розділу: «Expense Policy › Limits: …»	майже завжди: дешево і помітно покращує пошук

ГОЛОВНЕ

Чанк — одиниця пошуку і контексту. Розмір міряють у токенах; замалий втрачає зміст, завеликий несе шум. **Фіксоване вікно** просте, але ріже речення й таблиці. **Chunking за структурою** зберігає зміст цілим і знає розділ — для цього структуру треба зберегти ще під час витягування.

Глибше: Pinecone: Chunking strategies · Anthropic: Contextual Retrieval · tiktoken

---

РОЗДІЛ 7

7 Метадані, доступ і аудит

> Після розділу: ви знаєте, що зберігати поряд із текстом чанка і навіщо, чому права доступу перевіряють під час пошуку і що таке журнал подій.

7.1 Що таке метадані

Метадані — «дані про дані»: не сам текст, а інформація про нього. Це та сама картка бібліотекаря з розділу 1. Ось реальний запис одного чанка з практики:

```
{
  "chunk_id": "handbook/expense_policy.md#2",   # стабільний id: документ + номер чанка
  "source": "handbook/expense_policy.md",        # звідки — для цитування
  "title": "Expense Policy",
  "section": "Expense Policy > Limits",          # де саме в документі
  "access": ["all_employees"],                   # хто може бачити
  "effective": "2026-03-01",                     # дата чинності
  "doc_hash": "6eeb3732…",                       # хеш змісту документа
  "chunk_index": 2,
  "n_tokens": 66,
  "text": "| Category | Limit | Approval | …",
  "pipeline_version": "ingest-v1",               # яким кодом зроблено
  "ingested_at": "2026-10-28T14:13:00+00:00"
}
```

Кожне поле відповідає на конкретне питання, яке виникне пізніше:

Поле	Навіщо знадобиться
source, section	RAG показує користувачу джерело відповіді: «див. Expense Policy › Limits» (заняття 8)
effective	відфільтрувати застарілі документи або віддати перевагу новішим
access	не показати фінансовий звіт тому, кому не можна (7.2)
doc_hash, pipeline_version	зрозуміти, з якої версії документа і яким кодом зроблено чанк

7.2 Права доступу

У компанії не всі бачать усе: фінансовий звіт — лише фінансовий відділ. У практиці це задано файлом `access.json`: документи з папки `finance/` отримують `"access": ["finance"]`. Ключове питання — **де** перевіряти доступ.

Неправильно

Знайти чанки, передати моделі, а в промпті написати «не показуй фінансові дані, якщо користувач не з фінансів». Текст уже в контексті; одне вдале питання чи prompt injection (заняття 4) — і модель його перекаже.

Правильно

Під час пошуку фільтрувати чанки за групами користувача: чанк з `access: ["finance"]` взагалі не потрапляє в результати для інших. Модель не може розкрити те, чого не бачила.

Саме тому права доступу треба записати в метадані на етапі ingestion: пізніше, у векторній базі, за ними фільтруватимуть пошук (заняття 7).

7.3 Журнал подій (audit trail)

Коли користувач скаже «я шукаю договір з Baltic Freight, а система його не знає», треба швидко відповісти чому. **Audit trail** — журнал, куди пайплайн записує, що сталося з кожним файлом і чому:

```
{"event": "ocr_required",    "source": "scans/signed_nda_scan.pdf"}
{"event": "exact_duplicate", "source": "shared_drive/expense_policy (copy).md", "of":
"handbook/expense_policy.md"}
{"event": "near_duplicate",  "source": "policies/remote_work_policy.pdf", "kept":
"policies/remote_work_policy_v2.docx", "jaccard": 0.9}
```

З першого рядка видно відповідь: договір — це скан, він чекає на OCR. Без журналу довелося б дебажити пайплайн.

ГОЛОВНЕ

Метадані — картка кожного чанка: джерело, розділ, дата, доступ, версії. Вони потрібні для цитування, фільтрації і пояснення відповідей. **Права доступу перевіряють під час пошуку**, до моделі, а не інструкцією в промпті. Журнал подій пояснює, чому документа немає в пошуку.

РОЗДІЛ 8

8 Версіонування і продакшн-патерни

> Після розділу: ви розумієте, навіщо версіонувати датасет, і знаєте терміни, які зустрінете в документації та на співбесідах.

8.1 Версія датасету

На занятті 4 ми версіонували промпти: кожна зміна — нова версія, щоб будь-який результат можна було відтворити. З даними те саме. Якщо після зміни chunking якість відповідей упала, треба точно знати, з якою версією даних вона була кращою. Для цього пайплайн рахує **dataset_version** — хеш від manifest (усіх файлів з їхніми хешами) разом з версією пайплайна. Ті самі файли, оброблені тим самим кодом, завжди дають той самий id; змінився один файл або код — id інший.

Зміна chunking, очищення чи моделі embeddings — це **нова версія індексу**, і порівнювати її зі старою треба вимірюваннями (заняття 9), а не «на око». Для великих датасетів версії зберігають інструменти **DVC** і **lakeFS**, що працюють як git для даних.

8.2 Шари даних (medallion)

Поширений патерн — зберігати дані в кількох шарах, від сирих до готових:

Шар	Що там	Навіщо
Raw / bronze	оригінальні файли як є, ніколи не змінюються	можна переобробити з нуля новим кодом
Clean / silver	витягнутий і нормалізований текст + метадані документа	дедуплікація, перевірки якості
Chunks / gold	чанки з метаданими, готові до embeddings	індекс, який читає RAG

Головне правило — **raw ніколи не перезаписують**. Якщо завтра ви знайдете кращий спосіб чистити PDF, ви переобробите все з raw, а не правитимете індекс руками.

8.3 Терміни, які зустрінете

Термін	Що означає
ETL vs ELT	Extract-Transform-Load: перетворити дані до запису. Extract-Load-Transform: записати сирими, перетворювати потім. Для AI зазвичай ELT: raw зберігаємо, обробку можна повторити.
Connector / loader	Адаптер до конкретного джерела: Google Drive, Confluence, SharePoint, S3. Готові є в LlamaIndex, LangChain, Airbyte.
Upsert	Update + insert: вставити запис або оновити, якщо запис з таким id уже є. Основа ідемпотентності.
Tombstone	Позначка «документ видалено в джерелі». Без неї видалений файл назавжди лишається в індексі, а модель цитує скасовану політику.
Dead-letter queue	Окрема черга для файлів, які не вдалося обробити. Пайплайн не падає через один битий файл, а проблемні видно й можна повторити.
Backfill / reindex	Повна переобробка всіх історичних даних після зміни пайплайна або моделі embeddings.
Lineage	Ланцюжок походження: чанк → документ → файл → версія пайплайна. Відповідає на питання «звідки взялась ця відповідь?».
Freshness SLA	Домовленість, за скільки часу зміна в джерелі з'явиться в пошуку. Визначає вибір між batch і streaming.
Data contract	Домовленість з власником джерела про формат: які поля, кодування, як позначаються видалення. Зміна формату без попередження тихо ламає ingestion.
Layout-aware parsing	Парсер, який розуміє макет сторінки: колонки, таблиці, підписи до рисунків (Docling, unstructured, vision-моделі).

ГОЛОВНЕ

Дані версіонують так само, як промпти: **dataset_version** = хеш manifest + версія коду. Raw не перезаписують, а все інше можна перебудувати з нього. Видалення в джерелі обробляють явно (tombstone), а битий файл не має зупиняти весь пайплайн.

Глибше: Databricks: Medallion architecture · DVC · lakeFS

РОЗДІЛ 9

9 Перевірте себе

> Спробуйте відповісти без підглядання. Якщо питання викликає складність, поверніться до розділу в дужках.

1. Чому не можна просто передати моделі всі документи компанії в одному запиті? Назвіть дві причини. *(1.1, 6.1)*
2. Сервіс підтримки хоче, щоб новий тікет з'являвся в пошуку за хвилину, а HR-політики оновлюються раз на місяць. Який патерн ingestion ви оберете для кожного джерела і чому? *(2.2)*

3. Пайплайн упав на половині документів. Що має бути правдою, щоб його можна було просто запустити знову без дублів в індексі? *(2.3)*
4. PDF «успішно» оброблено, але в індексі з нього нуль символів. Що найімовірніше сталося і як це виявити автоматично? *(3.3)*
5. Чому DOCX не можна читати лише як список абзаців? Що втратиться? *(3.5)*
6. Що станеться, якщо файл у cp1251 прочитати як UTF-8? Як цього уникнути? *(3.6)*
7. Як автоматично знайти колонтитул у багатосторінковому PDF? Чому цей спосіб не спрацює на одній сторінці? *(4.3)*
8. Назвіть три речі, які не можна «чистити» з тексту, і поясніть чому. *(4.5)*
9. Два файли мають різні хеші байтів, але однаковий зміст. Який хеш їх об'єднає і що він робить з текстом? Для чого тоді хеш байтів? *(5.3)*
10. Чому зміна одного слова псує кілька шинґлів? Чому Jaccard двох версій великого документа все одно високий? *(5.4)*
11. Фіксоване вікно розрізало таблицю лімітів. Які дві проблеми це створює для пошуку і відповіді? *(6.3)*
12. Фінансовий звіт не має бачити ніхто, крім фінансового відділу. Де саме треба застосувати це правило і чому інструкція в промпті не підходить? *(7.2)*
13. Ви змінили розмір чанків, і якість відповідей упала. Що з розділу 8 допоможе повернутися до попереднього стану і довести, що причина саме в цьому? *(8.1)*

10 Перед заняттям

> На занятті працюємо з кодом з репозиторію курсу. Підготуйте його заздалегідь, щоб на занятті одразу запускати, а не налаштовувати.

Репозиторій і бібліотеки

```
git clone https://github.com/andbilous/ai-engineeering-2   # або git pull, якщо вже клонували
cd ai-engineeering-2/lesson-06-data-ingestion
pip install -r requirements.txt
```

Ключ API і Ollama цього разу не потрібні: весь пайплайн — код, без виклику моделі. Під час першого запуску `tiktoken` завантажує словник токенізатора (~1.7 МБ), тож потрібен інтернет. Python 3.10+.

Файл	Що це
`data/raw/`	9 документів вигаданої компанії Northwind Logistics: PDF зі шапками й переносами, скан, HTML з банером, два DOCX, Markdown і його копія, TXT у cp1251 — усі приклади з цього матеріалу
`data/access.json`	хто може читати документи з кожної папки (розділ 7.2)
`data/expected.json`	що має вийти: дублі, OCR, заборонені рядки, доступи
`starter/ingest.py`	пайплайн з TODO 1–10
`starter/check.py`	14 перевірок результату з номером TODO біля кожної

Домашня частина: TODO 1–5

Витягування й очищення робите вдома, а на занятті будуємо на ньому дедуплікацію, chunking, метадані й інкрементальний запуск. Пишіть код з AI-асистентом, але кожен рядок маєте вміти пояснити: на занятті хтось із вас покаже свій розв'язок.

TODO	Що зробити	Розділ
1	TXT / MD: визначити кодування замість «завжди UTF-8», прибрати BOM	3.6
2	HTML: лишити статтю, прибрати меню, cookie-банер, скрипти й футер; заголовки → `#`	3.4
3	PDF: текст по сторінках через `\f`; позначити скан без текстового шару як `needs_ocr`	3.2, 3.3
4	DOCX: зберегти заголовки, нумеровані списки й таблиці (як рядки `| a | b |`)	3.5
5	`normalize()`: NFKC, невидимі символи, колонтитули й номери сторінок, переноси, пробіли	4.2–4.4

TODO 6–10 (дедуплікація, chunking, метадані, manifest) робимо на занятті. Запуск:

```
python starter/ingest.py
python starter/check.py
```

Що побачите

```
FOR CHAT: files 9 | indexed 8 | chunks 20 | skipped unchanged 0 | dataset
PASS  TODO 1: legacy cp1251 file decoded
…
FAIL  TODO 6: exact duplicate dropped
6/14 checks passed
```

Без змін у коді буде `0/14`. Після TODO 1–5 — `6/14`: перевірки 1–5 проходять, а 6–10 чекають на заняття. Порожній `dataset` — теж нормально, це TODO 10.